ISO/IEC JTC 1/SC 22/WG 21 P2081R0
Date: 2020-01-13
Audience: LEWG, LWG
Thomas Köppe <tkoeppe@google.com>

Rebase the Library Fundamentals v3 TS on C++20

Abstract

The Library Fundamentals v3 TS should be based on C++20.

Contents

Proposal

The working draft of the Extensions for Library Fundamentals, Version 3 Technical Specification ("LFTSv3" or just "LFTS" for short) is currently (as of P0996R1) based on the published C++ standard, i.e. C++17. It is very unlikely, and indeed not planned at this point, that we will publish the TS before the C++20 DIS has been published (which we expect to happen at the next meeting as of the time of writing, which is Prague 2020). Therefore, we have the option of basing the TS on the C++20 DIS, and we should do this to allow us to remove facilities that have been merged into the main standard, and to take advantage of new language facilities.

If the C++20 IS is published before we complete the PDTS ballot for the LFTS, we should update the LFTS working draft to refer to the IS instead of the DIS. The acceptance of this proposal should include approval of such a future change, which would then be an editorial matter to align the wording with the design intent.

Proposed wording

Front matter updates

Modify [1.2, general.references] paragraphs 1, 2, and 3 as follows:

> **1.2 Normative references [general.references]**
>
> 1. The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
>
> - ISO/IEC ~~14882:2017~~14882:–, Programming Languages — C++
>
> [*Note*: Under preparation. Stage at the time of publication: ISO/DIS 14882:2020. — *end note*]
>
> 2. ISO/IEC ~~14882:2017~~14882:– is herein called the *C++ Standard*. References to clauses within the C++ Standard are written as "~~C++17~~C++20 §3.2". The library described in ISO/IEC ~~14882:2017 clauses 20–33~~14882:– clauses 16–32 is herein called the *C++ Standard Library.*
>
> 3. Unless otherwise specified, the whole of the C++ Standard's Library introduction (~~C++17 §20~~C++20 §16) is included into this Technical Specification by reference.

Modify [1.3, general.namespaces] paragraphs 2:

> 2. Each header described in this technical specification shall import the contents of `std::experimental::fundamentals_v3` into `std::experimental` as if by
>
> ```
> ~~namespace std::experimental {~~
> ~~  inline namespace fundamentals_v3 {}~~
> ~~}~~
> namespace std::experimental::inline fundamentals_v3 {}
> ```

Deletions of merged material

Delete subclause [6.1, container.erasure].

~~**6.1 Uniform container erasure [container.erasure]**~~

~~**6.1.1 Header synopsis [container.erasure.syn]**~~

~~1. For brevity, [...]~~

~~[...] are intentionally not provided. – *end note*~~]

Delete clause [11, reflection].

~~**11 Reflection library [reflection]**~~

~~[...]~~

Delete from Table 1 — C++ library headers:

```
<experimental/algorithm>
<experimental/array>
<experimental/deque>
<experimental/forward_list>
<experimental/functional>
<experimental/future>
<experimental/iterator>
<experimental/list>
<experimental/map>
<experimental/memory>
<experimental/memory_resource>
<experimental/propagate_const>
<experimental/random>
<experimental/scope>
<experimental/set>
<experimental/source_location>
<experimental/string>
<experimental/type_traits>
<experimental/unordered_map>
<experimental/unordered_set>
<experimental/utility>
<experimental/vector>
```

Delete from Table 2 — Significant features in this technical specification:

Doc. No.	Title	[...]
N4388	A Proposal to Add a Const-Propagating Wrapper to the Standard Library	[...]
[...]	[...]	[...]
~~N4273~~	~~Uniform Container Erasure~~	~~[...]~~
[...]	[...]	[...]
~~N4519~~	~~Source-Code Information Capture~~	~~[...]~~

Systematic updates of the method of specification and presentation

Editorially apply the new compact inline namespace syntax wherever it applies.

> **3.1.1 Header `<experimental/utility>` synopsis [utility.syn]**
>
> ```
> #include <utility>
>
> namespace std::experimental::inline fundamentals_v3 {
> ~~inline namespace fundamentals_v3 {~~
>
> // 3.1.2, Class erased_type
> struct erased_type { };
>
> ~~} // namespace fundamentals_v3~~
> } // namespace std::experimental::inline fundamentals_v3
> ```
>
> [Further edits not shown.]

Update section numbers, and update the “C++17-concept” names everywhere and replace “satisfies” with “meets”, e.g. “satisfies the requirements of `CopyConstructible`” becomes “meets the *C++17CopyConstructible* requirements”.

> **~~20.7.7~~20.10.8 uses_allocator [allocator.uses]**

> ~~20.7.7.1~~20.10.8.1 **uses_allocator trait [allocator.uses.trait]**
>
> `template <class T, class Alloc> struct uses_allocator;`
>
> 1. *Remarks*: Automatically detects whether `T` has a nested `allocator_type` that is convertible from `Alloc`. Meets the ~~BinaryTypeTrait~~*C++17BinaryTypeTrait* requirements (~~C++17 §23.15.1~~C++20 §20.15.1). The implementation shall provide a definition [...]
>
> [Further edits not shown.]

Update C++17-style *Requires*/*Remarks*/*Postconditions* into *Mandates*/*Expects*/*Constraints*/*Ensures*.

> **3.2.2.3 propagate_const constructors [propagate_const.ctor]**
>
> [*Note*: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — *end note*]
>
> `template <class U> see below constexpr propagate_const(propagate_const<U>&& pu);`
>
> *Remarks*: ~~This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>`.~~ The constructor is specified as explicit if and only if `!is_convertible_v<U&&, T>`.
>
> *Constraints*: `is_constructible_v<T, U&&>` is `true`.
>
> *Effects*: Initializes `t_` as if direct-non-list-initializing an object of type `T` with the expression `std::move(pu.t_)`.
>
> [Further edits not shown, will be included in a future revision.]